ClinProg-Bench: A Public Benchmark for Clinical-Trial Statistical Programming Automation

Anonymous Author(s)
Affiliation
Address
email

Abstract

Clinical-trial statistical programming—generating CDISC-conforming SDTM and ADaM datasets, regulatory tables, and validation artefacts—is a high-stakes, domain-specialised software task absent from existing code-LLM benchmarks. We introduce **ClinProg-Bench**, a public benchmark of **250** expert-curated tasks across five categories (code generation, code review, specification extraction, documentation, and debugging), grounded in the publicly redistributable CDISC SDTM/ADaM Pilot Project and scored by deterministic, language-agnostic oracles. The benchmark introduces a stateful runner contract that supports multi-agent and retrieval-augmented agents, a two-channel leakage audit (SHA-256 byte overlap + 13-gram Jaccard, both reporting zero hits), and a dual-reviewer human audit on a stratified random subset (**Cohen's** $\kappa \geq 0.8$). Three frontier models evaluated under zero-shot and fixture-grounded conditions achieve macro scores of 0.22–0.28, with no model exceeding 0.35 on code generation and scoring near zero on log review and debugging—establishing substantial headroom for future systems. We release the corpus under CC-BY-4.0, the toolkit under Apache-2.0, and a Zenodo-archived snapshot (DOI withheld for double-blind review).

1 Introduction

Clinical-trial statistical programming sits at the intersection of regulated software, domain-specific data standards (CDISC SDTM, ADaM, Define-XML), and high-stakes outputs: every dataset and listing in a regulatory submission is inspected by FDA, EMA, or PMDA reviewers and feeds directly into approval decisions. The work is also voluminous—a single phase-III trial typically yields hundreds of programs across SAS, R, and increasingly Python—and expensive: median compensation for clinical statistical programmers exceeds $120k/year and the global shortage of qualified personnel is well documented.

Despite recent advances in code generation, applying general-purpose LLMs to clinical programming surfaces systematic failure modes that existing benchmarks do not capture. Consider two representative errors observed when prompting frontier models on CDISC tasks: (1) a model asked to derive the SDTM AE domain populates `AEBODSYS` with the verbatim term instead of the MedDRA System Organ Class, producing a dataset that passes functional tests but violates CDISC controlled terminology; (2) a model generating an ADaM ADSL dataset assigns `TRT01PN` values of `1, 2, 3` instead of the protocol-specified `0, 54, 81`, introducing a treatment-mapping error that would propagate through every downstream efficacy table. These failures are invisible to general code benchmarks but would trigger regulatory rejections. ClinProg-Bench is designed to expose precisely this class of domain-specific correctness violations.

No existing public benchmark evaluates code LLMs or agentic systems on clinical-programming workflows. General code benchmarks (HumanEval, MBPP, SWE-Bench, BigCodeBench) cover

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

algorithmic Python or open-source maintenance; biomedical NLP benchmarks (MedQA, PubMedQA) target text understanding rather than code; and pharma-internal evaluations rely on proprietary studies that cannot be released. The vacuum forces practitioners to evaluate models on hand-rolled, non-comparable test suites.

Contributions.

1. **ClinProg-Bench** (§3): 250 expert-curated tasks spanning code generation, code review, specification extraction, documentation, and debugging, all grounded in the publicly redistributable CDISC SDTM/ADaM Pilot Project (CC-BY-4.0).
2. **Deterministic scorers** (§4): five category-specific scorers using AST equivalence, set match, confusion matrix, slot fill, and patch-apply rubrics. No LLM-as-a-judge.
3. **Stateful runner contract** (§4): a language-agnostic workspace protocol that admits multi-agent and RAG-augmented systems, not just single-turn inference.
4. **Governance scaffolding** (§3): two-channel leakage audit and a Cohen's $\kappa \geq 0.8$ dual-reviewer audit on a stratified random subset.
5. **Public release**: Apache-2.0 / CC-BY-4.0, Zenodo DOI, CI-tested on every push, and a rules-only reference baseline for scorer validation.

2 Related Work

Code-generation benchmarks. HumanEval [4], MBPP [1], and APPS [8] evaluate algorithmic Python on isolated functions; BigCodeBench [22] extends the paradigm to diverse function calls and complex instructions. SWE-Bench [10] and its agent-driven successor SWE-Agent [19] target repository-level bug fixing within open-source Python projects; RepoBench [15] benchmarks cross-file code completion; LiveCodeBench [14] evaluates on continuously updated competitive-programming problems to reduce data contamination. These benchmarks share our emphasis on deterministic scoring—unit tests, exact match, AST equivalence—but address general-purpose programming. None cover SAS or R in regulated settings, domain-specific data standards such as CDISC SDTM/ADaM [2, 3], or the documentation and validation artefacts required in regulatory submissions. ClinProg-Bench fills this gap by grounding all tasks in a real clinical-trial substrate and scoring against CDISC-conforming gold outputs.

Domain-specific code benchmarks. DS-1000 [13] evaluates data-science code generation across NumPy, Pandas, and TensorFlow with a "strict" mode that rejects syntactic variants; Spider [20] benchmarks text-to-SQL semantic parsing against gold query sets; CrossCodeEval [5] measures cross-file code completion across six programming languages; CRUXEval [7] probes code reasoning and execution understanding. These works share our emphasis on grounded, deterministic evaluation but address different domains without regulatory constraints. A key distinction is that ClinProg-Bench tasks produce outputs that must conform to externally imposed standards (CDISC controlled terminology, ADaM variable-naming conventions) rather than satisfying only functional correctness. A program that produces numerically correct output may still fail evaluation if variable names, metadata, or documentation do not comply with the submission standard—a qualitatively different acceptance criterion.

Biomedical and clinical evaluations. MedQA [11], PubMedQA [12], and MedAlign [6] probe clinical and biomedical reasoning over natural language and structured text. These benchmarks evaluate text understanding and clinical decision-making but do not assess code generation, data-standards conformance, or the software engineering pipeline that produces regulatory datasets and tables. Industry standards bodies—CDISC [2, 3], PHUSE, and the ICH [9]—define the data models, controlled terminologies, and process frameworks (GCP) that clinical programmers work within, but they provide substrate and guidance rather than benchmarking infrastructure. ClinProg-Bench bridges this gap: it is the first public benchmark that evaluates AI systems on the full clinical-programming lifecycle (code generation, review, specification extraction, documentation, debugging) against a CDISC-conforming substrate released under an open license.

Agentic benchmarks. GAIA [17] evaluates general-purpose AI assistants on tasks requiring web browsing, tool use, and multi-step reasoning; OSWorld [18] benchmarks multimodal agents in real desktop environments; WebArena [21] tests end-to-end web interaction; AgentBench [16] provides a multi-environment evaluation suite across operating systems, databases, and web tasks. These benchmarks demonstrate the value of stateful, multi-step evaluation protocols for agentic systems. ClinProg-Bench adapts this philosophy to the regulated software domain: its stateful runner contract (§4) supports multi-agent systems that can read specifications, write programs, inspect logs, and iterate—mirroring the real clinical-programming workflow. Unlike general agentic benchmarks, ClinProg-Bench evaluates domain-specific competence with deterministic oracles, avoiding the reliability and reproducibility concerns of LLM-as-a-judge scoring.

3 Dataset

3.1 Substrate and provenance

All tasks are grounded in the publicly redistributable CDISC SDTM/ADaM Pilot Project (CC-BY-4.0), pinned via Git submodule at commit `658fcc05506b169a27dee6e2c3a1ccdaaf64a716`. Each task's `provenance` block records its derivation script and the substrate commit, making generation end-to-end auditable. A SHA-256 manifest of 272 fixture files is committed alongside the tasks.

Vendored fixture subset. For grounding-conditioned baselines (§5), a 26-file, 682 KB subset of the substrate is vendored verbatim under `fixtures/`. The subset is the intersection of the 67 fixture base paths referenced by `inputs.fixtures` across the 250 task specifications and the files that exist in the v1.0.0 substrate; the remaining 42 references (15 SDTM domain-creator R programs and 27 TLF mock templates with hyphen/underscore naming drift) are unauthored upstream and are tracked as v0.1.1 work (§6). Vendoring is verbatim with no modification; substrate `LICENSE` and `NOTICE.md` accompany the files.

3.2 Composition

ClinProg-Bench contains **250** tasks distributed across five categories (Table 1). Each task is a validated `TaskSpec` JSON file bundled with fixture files, a markdown specification, and a gold output under `gold/<task_id>/`.

Table 1: Task taxonomy. Output kinds and scorers are fixed per sub-category to support deterministic evaluation.

Category	Sub-categories	N	Output kind	Scorer
T1 Code Generation	SDTM (22), ADaM (18), TLF (20)	60	program	`codegen`
T2 Code Review	program (25), validation (25)	50	JSON	`log_review`
T3 Spec Extraction	SDTM (21), ADaM (18), TLF (11)	50	JSON	`spec_extract`
T4 Documentation	TLF (16), dataset card (13), synopsis (11)	40	text	`doc`
T5 Debugging	SDTM (17), ADaM (17), TLF (16)	50	patch	`debug`
Total		250		

3.3 Quality assurance

Schema validation. Every task is parsed by the strict pydantic schema in `src/clinprog_bench/schema.py`; `task_id` is fixed to the regex `T[1-5]\.[1-9][0-9]?\.[a-z0-9_]+\.[0-9]{3}` and `seed_commit` to a 40-char SHA-1. Validation runs on every CI push (`.github/workflows/eval.yml`, job `validate`).

Leakage audit. A two-channel audit (`scripts/leakage_audit.py`) checks (a) SHA-256 byte overlap between gold outputs and the substrate, and (b) 13-gram Jaccard similarity between task prompts and substrate text. Both channels report **0 hits** on the v0.1.0 release; the report is committed at `docs/leakage-audit.md`.

Dual-reviewer audit. A stratified random subset (seed `20260426`, 4 tasks per category, 20 total) is independently rated by two domain experts using a 5-point rubric. The acceptance threshold is Cohen's $\kappa \geq 0.8$. Materials are shipped under `review_package/` and the audit log template is at `docs/audit-log.md`.

3.4 Licensing and release

The benchmark toolkit (Python package, scorers, runner contract) is released under **Apache-2.0**. The task corpus (JSON, gold outputs, fixtures) is released under **CC-BY-4.0**, inheriting the substrate license. A versioned snapshot is archived on Zenodo (DOI withheld for double-blind review). A full Gebru-style data card lives at `DATA_CARD.md` in the repository root.

4 Benchmark Design

4.1 Runner contract

ClinProg-Bench specifies a language-agnostic *runner contract* (`src/clinprog_bench/runners/contract.md`, v0.1.0) that any agent must satisfy. For each task the harness:

1. materialises a fresh workspace populated with the task's fixtures,
2. invokes the agent's `BaseAdapter.run(task, workspace)` method,
3. collects a `Submission` record (immutable; outputs keyed by the task's `output_kind`),
4. passes the submission to the category-specific scorer.

The contract is intentionally state*ful*: the workspace persists across sub-tasks within a multi-step task, and the contract admits multi-agent and retrieval-augmented systems alongside single-turn LLMs.

4.2 Scorers

Each category has a single deterministic scorer; *no* LLM-as-a-judge is used. Scorers are pure functions of (gold output, submission), implemented in `src/clinprog_bench/scorers/`, and unit-tested (`tests/scorers/`, 79 tests, 100% pass on v0.1.0).

Table 2: Scoring rubrics per category.

Scorer	Method	Output range
`codegen`	AST-equivalence + slot-fill on key statements	$[0, 1]$
`log_review`	set match / confusion matrix on review findings	$[0, 1]$
`spec_extract`	set match on extracted slots	$[0, 1]$
`doc`	slot-fill on required documentation fields	$[0, 1]$
`debug`	patch-apply: does the patch fix the seeded defect?	$\{0, 1\}$

The aggregate benchmark score is the macro-average across the five categories, reported alongside per-category and per-sub-category breakdowns.

4.3 Reference baseline

A rules-only `ReferenceBaselineAdapter` (`src/clinprog_bench/runners/baselines/reference.py`) emits empty or trivially-formed submissions. Its purpose is *not* comparison but scorer validation: the reference baseline scores 0.00 across all categories, confirming that no scorer admits trivial outputs.

4.4 Evaluation harness and CI

The benchmark CI workflow (`.github/workflows/eval.yml`) runs three jobs on every push to `master`: schema-lint, unit tests, and a reference-baseline smoke evaluation. A successful CI run is a precondition for tagging a release.

5 Baselines

5.1 Setup

We evaluate two classes of baselines under the runner contract of §4:

1. **Rules-only reference** (`reference-baseline`): a deterministic adapter that emits trivially-formed submissions, run on all 250 tasks. It validates that no scorer admits empty answers and establishes the deterministic floor.
2. **Single-turn LLM** (`deepseek-chat`, `claude-sonnet-4.5`, `glm-4.5`): the task prompt and category-specific output instructions are passed to a frontier model in one call; the parsed response is scored as a submission. Each model is run under two grounding conditions: (i) *zero-shot*, prompt only; (ii) +*grounding*, with vendored CDISC fixtures (§3) inlined into the prompt under an 8 KB-per-request budget.

A retrieval-augmented agent (BM25 over the same fixture subset) is deferred to v0.1.1 and discussed in §6.

5.2 Decoding and budget

All LLM baselines use temperature 0, `top_p`$= 1$, and a 4096-token generation cap. Each LLM is evaluated on a stratified subset of 25 tasks (5 per category, seed `20260427`); the reference baseline runs on all 250. Wall-clock latency for the 25-task pass with grounding was 422 s for DeepSeek-Chat, 519 s for Claude Sonnet 4.5, and 1325 s for GLM-4.5; GLM-4.5 incurred 7 upstream read timeouts on long-prompt tasks that were scored as zero-output submissions.

5.3 Results

Table 3 reports the macro-average score per baseline under both conditions; per-task submissions and the live HTML leaderboard are at `docs/leaderboard/`.

Table 3: Baseline results on ClinProg-Bench v0.1.0. Per-category columns are reported for the +*grounding* condition. "ZS" denotes the zero-shot macro for comparison. LLM rows are evaluated on a stratified 25-task subset (5 per category, seed `20260427`); the reference row covers all 250 tasks. Δ = +grounding $-$ zero-shot.

Baseline	T1	T2	T3	T4	T5	Macro	ZS	Δ
reference-baseline[‡]	–	–	–	–	–	–	0.05	–
deepseek-chat	0.35	0.00	0.44	0.56	0.05	**0.28**	0.27	$+0.01$
claude-sonnet-4.5	0.25	0.00	0.44	0.56	0.00	0.25	0.26	-0.01
glm-4.5	0.10	0.00	0.08	0.44	0.07	0.14[†]	0.22	-0.08[†]

[†] Includes 7 upstream read timeouts on long-prompt tasks; the operational drop reflects provider-side context handling under fixture inlining, not raw capability.
[‡] Reference baseline emits trivially-formed submissions and does not consume fixtures; only the zero-shot column applies. Per-category zero-shot scores are T1 $= 0.25$, T2 $= 0$, T3 $= 0$, T4 $= 0$, T5 $= 0$.

5.4 Discussion

Four patterns are visible in Table 3. First, the category ordering is stable across grounding conditions: T4 (documentation) and T3 (specification extraction) are the easiest, T1 (code generation) is harder, and T2 (log review) and T5 (debug patches) remain near 0 even with grounding—these categories require the specific SAS-log and `_v2` program fixtures that the v0.1.0 substrate does not yet author (§3). Second, frontier models cluster within roughly 5 macro-points of one another in the zero-shot setting (0.22–0.27), suggesting that prompt engineering and grounding budgets—not raw capability—dominate at v0.1.0. Third, fixture grounding does *not* uniformly help: DeepSeek-Chat gains $+0.01$ (T1 lifts from 0.20 to 0.35 once spec documents are inlined), Claude Sonnet 4.5 is unchanged within noise, and GLM-4.5 loses 0.08 macro-points operationally because longer prompts push tail latencies past the provider's read-timeout. We report the operational number rather than retrying; see §6 for

the implications and the planned mitigation. Fourth, the identical T4 score (0.56) across DeepSeek and Claude is a scorer ceiling, not chance: the documentation rubric awards partial credit for length and citation structure that any competent LLM satisfies, but the strict "pass" threshold requires evidence-grounded claims that none of the three models produces under either condition. Closing this gap is the focus of the retrieval-augmented baseline planned for v0.1.1.

6 Limitations

One substrate. ClinProg-Bench v0.1.0 is grounded entirely in the CDISC SDTM/ADaM Pilot Project (a phase-II Alzheimer's-disease protocol). This bounds external validity: results on Pilot01-derived tasks may not generalise across therapeutic areas, study phases, or proprietary corporate standards. A v0.2 release with at least one additional public substrate is planned.

Scorer rigidity. Deterministic scorers reject semantically equivalent but syntactically divergent answers (e.g. a SAS `IF/THEN` that achieves the same derivation as a `WHERE` clause). Slot-fill and AST-equivalence rubrics mitigate but do not eliminate this. We recommend reporting scorer-conformance error rates alongside the headline number.

Static gold outputs. Gold outputs are frozen at substrate commit `658fcc05`; downstream regulatory practice (CDISC versions, FDA guidance) evolves. The benchmark is versioned and Zenodo-archived to make this drift explicit.

Single English locale. All prompts and specifications are in English. Multilingual clinical-programming evaluation is left to future work.

Partial fixture coverage. 42 of the 67 fixture base paths referenced by task specifications point at files unauthored in the v1.0 substrate (15 SDTM `create_*.R` programs for the CM, DS, EX, LB, MH, QS, SC, SV domains, and 27 TLF mock templates including hyphen/underscore naming-style drift). The runner silently skips unresolvable paths, so the no-grounding baseline is unaffected, but the +grounding macro is upper-bounded by a 40% coverage gap on T2/T5. Authoring these fixtures and a BM25 retrieval-augmented baseline are the headline items for v0.1.1.

Operational variability under grounding. GLM-4.5 with fixture inlining incurred 7 upstream read-timeout errors out of 25 prompts, dropping its operational macro by 0.08 (§5). The benchmark records the operational outcome rather than retrying because submission throughput under longer prompts is itself a property worth measuring; submitters should report timeouts and retries explicitly.

7 Broader Impact and Ethics

No human-subjects data. All fixtures derive from the CDISC SDTM/ADaM Pilot Project, a public reference dataset distributed by CDISC for educational use. The dataset contains no real patient records, no protected health information, and no proprietary trial data. No IRB review is required.

Intended use. ClinProg-Bench is intended to evaluate code-LLMs and agentic systems on clinical-programming tasks under reproducible conditions. It is *not* a certification of fitness for regulatory submission: a model scoring well on this benchmark is not thereby cleared for production use in a phase-III trial. Practitioners should continue to follow ICH E6 (R3), 21 CFR Part 11, and their organisation's validation SOPs.

Misuse considerations. The benchmark could in principle be abused to over-claim AI capability in regulatory contexts. We mitigate this with: (a) explicit benchmark-vs-deployment scoping in this section, (b) a deterministic, fully-public evaluation harness that resists cherry-picked reporting, and (c) a leakage audit and dual-reviewer audit committed to the repository.

Author qualifications and review. The dataset was authored by the listed author with the CDISC Pilot Project as substrate; a three-member expert panel (clinical statistical programmers in the listed acknowledgements) audits a stratified subset under a Cohen's $\kappa \geq 0.8$ acceptance criterion (§3).

Carbon footprint. v0.1 LLM baseline runs (§5) total under 5 M tokens and complete in well under one GPU-hour-equivalent; the benchmark itself adds negligible compute cost. Future evaluations will report token counts and wallclock budgets in the leaderboard.

Maintenance. The author commits to maintaining the benchmark through the next two NeurIPS review cycles. Release cadence, deprecation policy, and contact channels are documented in the repository `README.md`; the Zenodo DOI provides long-term archival.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] CDISC. CDISC SDTM implementation guide v3.4. `https://www.cdisc.org/standards/foundational/sdtmig`, 2023. Accessed 2026-04.

[3] CDISC. CDISC ADaM implementation guide v1.3. `https://www.cdisc.org/standards/foundational/adam`, 2024. Accessed 2026-04.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Yangruibo Ding et al. CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion. *NeurIPS*, 2023.

[6] Scott L. Fleming et al. MedAlign: A clinician-generated dataset for instruction following with electronic medical records. *AAAI*, 2024.

[7] Alex Gu et al. CRUXEval: A benchmark for code reasoning, understanding, and execution. In *ICLR*, 2024.

[8] Dan Hendrycks, Steven Basart, Saurav Kadavath, et al. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks*, 2021.

[9] International Council for Harmonisation. ICH harmonised guideline: Integrated addendum to E6(R1): Guideline for good clinical practice E6(R2). `https://www.ich.org/page/efficacy-guidelines`, 2016. Accessed 2026-04.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[11] Di Jin, Eileen Pan, Nassim Oufattole, et al. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 2021.

[12] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *EMNLP*, 2019.

[13] Yuhang Lai, Chengxi Li, Yiming Wang, et al. DS-1000: A natural and reliable benchmark for data science code generation. In *ICML*, 2023.

[14] Changshu Liu et al. LiveCodeBench: Holistic and contamination free evaluation of code large language models. *arXiv preprint arXiv:2403.07974*, 2024.

[15] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2024.

[16] Xiao Liu et al. AgentBench: Evaluating large language models as agents. In *ICLR*, 2024.

[17] Grégoire Mialon, Clémentine Fourrier, Craig Swift, et al. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

[18] Tianbao Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *NeurIPS*, 2024.

[19] John Yang et al. SWE-agent: Agent–computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[20] Tao Yu, Rui Zhang, Kai Yang, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *EMNLP*, 2018.

[21] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, et al. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024.

[22] Terry Yue Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

NeurIPS Paper Checklist

1. **Claims**
   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] Abstract and introduction clearly state the 250-task, 5-category benchmark scope and the three-model baseline evaluation (§5).
2. **Limitations**
   (a) Does the paper discuss the limitations of the work performed by the authors? [Yes] See §6.
3. **Theory**
   (a) Does the paper provide theoretical analysis? [NA] This is a benchmark/dataset paper.
4. **Experiments**
   (a) Does the paper include experiments on datasets or benchmarks? [Yes] Baseline results on ClinProg-Bench v0.1.0 (§5, Table 3).
   (b) Is there enough information to reproduce the main experimental results? [Yes] Appendix A provides one-command instructions. Code, data, and CI workflow are included in the supplementary package.
   (c) Does the paper report error bars or statistical significance? [No] Baselines are deterministic (temperature 0); no stochasticity to report.
5. **Compute**
   (a) Does the paper report the computational budget (e.g., GPU hours, cloud cost)? [Yes] Appendix A reports per-baseline API costs and wall-clock time.
6. **Code and Data**
   (a) Does the submission include code for the proposed method or system? [Yes] Full benchmark toolkit (`src/clinprog_bench/`), 5 scorers, runner contract, and CI workflow included in the supplementary ZIP.
   (b) Does the submission provide sufficient details to reproduce the code? [Yes] Appendix A; `pyproject.toml` pins all dependencies; 79 tests pass.
7. **Datasets**
   (a) Does the paper describe the dataset creation process? [Yes] See §3 for provenance, construction, and quality assurance.
   (b) Does the paper include a Datasheet or Data Card? [Yes] `DATA_CARD.md` in the repository root follows Gebru et al. (§3).
   (c) Does the paper provide Croissant metadata? [Yes] `croissant.json` committed at the repository root, following the MLCommons Croissant specification (two record sets: `tasks` and `gold`).
   (d) Is the dataset hosted with a persistent identifier? [Yes] Zenodo archive with DOI (withheld for double-blind review).
8. **Licensing**
   (a) Does the paper specify the license for the code and data? [Yes] Code: Apache-2.0. Data: CC-BY-4.0. Both committed as `LICENSE` and `LICENSE-DATA` (§7).

9. **Ethics**

   (a) Does the paper discuss potential negative societal impacts? [Yes] See §7.

   (b) Does the paper describe safeguards for sensitive data? [NA] No human-subjects data; substrate is the public CDISC Pilot Project with synthetic patients.

10. **Human Subjects**

   (a) Does the paper involve human subjects or participants? [No] The benchmark uses the public CDISC Pilot01 synthetic dataset; no IRB approval required.

A Reproducibility

This appendix provides one-command instructions for reproducing the baseline experiments reported in Table 3.

A.1 Environment setup

```
# Clone and install (Python >= 3.11)
git clone https://github.com/anonymous/clinprog-bench.git
cd clinprog-bench
pip install .                    # or: uv sync

# Verify: schema validation + unit tests
clinprog-bench validate tasks/
pytest -q                        # 79 tests, <1s
```

A.2 Reference baseline (all 250 tasks)

```
python -m clinprog_bench.runners.reference_baseline
# Output: results/reference_baseline_<timestamp>.json
# Expected macro zero-shot: 0.05
```

A.3 LLM baselines (25-task stratified subset)

Requires API keys set as environment variables: `DEEPSEEK_API_KEY`, `ANTHROPIC_API_KEY`, `GLM_API_KEY`.

```
# Run all three LLM baselines (both conditions)
python scripts/run_llm_baseline.py \
    --models deepseek-chat claude-sonnet-4-5 glm-4-5 \
    --subset-seed 20260427 --subset-size 25 \
    --conditions zero-shot grounding \
    --output results/

# Results are written per-model to results/<model>_<condition>.json
```

A.4 Continuous integration

The GitHub Actions workflow (`.github/workflows/eval.yml`) runs on every push and pull request. It executes three jobs sequentially: (1) **schema-lint**: validates all 250 task JSONs, runs `ruff`, `black`, and `mypy`; (2) **unit**: runs the full `pytest` suite; (3) **baseline**: smoke-tests the reference baseline runner. The CI badge in the repository README reflects the latest master status.

A.5 Hardware and cost

All baseline experiments were run on a consumer laptop (Intel i7-12700H, 32 GB RAM, no GPU) with API calls to external model providers. Per-baseline API costs were approximately: DeepSeek-Chat \$0.12 (25 tasks $\times$ 2 conditions), Claude Sonnet 4.5 \$1.80, GLM-4.5 \$0.08. Total wall-clock time: $\sim$40 minutes for the full 25-task three-model matrix.